Burntwax FPS Demo

Complete Documentation

Version 1.0 | Unity 6+ | Burntwax Collective

Table of Contents

1. Overview & Features 3
    - What's Included 3
    - What Makes This Different 3
    - Demo Content vs. Framework 4
2. Requirements & Compatibility 5
3. Installation & Quick Start 6
    - Step 1: Import the Package 6
    - Step 2: Open the Main Menu Scene 6
    - Step 3: Press Play 6
    - Step 4: Explore the Playground 6
    - Step 5: Experiment with Variables 6
4. Folder Structure 7
5. Architecture Overview 8
    - The Hierarchical State Machine Pattern 8
    - System Architecture 8
    - How the Three State Machines Connect 8
    - What Skill Level Do I Need? 9
6. Player Movement 10
    - 6.1 State Machine Concepts 10
        - Root States 10
        - Sub States 10
        - Code Difference: Root vs. Sub State 10
        - The PlayerStateMachine Script 11
    - 6.2 Adding a New State 11
7. Cameras 12
    - Included Cameras 12
    - Adding a New Camera 12
8. Gun System 13
        - Config Reference 13

8.5 Specialized Mechanics 15
Incremental Reloading (Shotgun-style) 15
Procedural Recoil (SMG-style) 15
9. Impact Effects 16
Included Surface Effects 16
Creating a New Impact Effect 16
10. Save & Load System 17
Basic Usage 17
What Gets Saved 17
Adding New Saveable Objects 17
11. Adding New Arms (Advanced) 18 19
12. Modifying Inputs 20
Changing Existing Bindings 20
Adding New Inputs 20
13. Pause Menu & UI 20
14. Troubleshooting & FAQ 21
"I installed a new package and now I get missing reference errors" 21
"My new gun doesn't switch properly / wrong gun equips" 21
"Arms are invisible at certain camera angles" 21
"Save/Load breaks when switching between Editor and Build" 21
"Objects I added don't save/load" 21
"Seesaw or moving slopes don't work" 21
"Something isn't working and I can't figure out why" 21
15. Known Limitations 21
16. Glossary 22
17. Support & Contact 22
18. Changelog 23
Version 1.0 — Initial Release 23

1. Overview & Features

The Burntwax FPS Demo is a production-ready first-person shooter foundation built in Unity. It provides a modular, extensible set of systems for player movement, camera control, weapons, impact effects, and save/load — all built on a consistent hierarchical state machine architecture.

The framework is designed so you can start with the included demo scene and build your game on top of it, rather than trying to bolt systems into an existing project. Every major system uses the same state machine pattern, so once you learn how one works, the rest follow the same logic.

What's Included

System	Description
Player Movement	Full FPS character controller with walking, sprinting, jumping, falling, crouching, wall running, and slope handling. Built on a hierarchical state machine with root states and sub states.
Camera System	Cinemachine-based FPS cameras with separate virtual cameras for normal view, aiming (reduced FOV), and extensible custom states. Tied to the same state machine structure as player movement.
Gun System	Scriptable Object-driven weapon framework supporting multiple gun types. Includes configurations for shooting, ammo, audio, damage, trails, crosshairs, and camera recoil. Supports incremental reloading (shotgun-style) and procedural recoil (SMG-style). Pickup-based acquisition system.
Impact Effects	Tag-based surface effect system using Scriptable Objects. Assign any material type (metal, glass, wood, etc.) unique VFX and randomized audio on bullet impact.
Save & Load	Persistent save system that tracks player state, gun inventory, ammo, object positions, and rigidbody states. Supports adding custom object types through the ObjectManager.
Input System	Built on Unity's new Input System with a centralized InputManager for easy rebinding and extension.
Pause Menu	Functional pause menu with resume and main menu navigation.
Demo Scene	Playground scene with gun pickups, destructible crates, ammo/health pickups, targets, and varied terrain for testing all systems.

What Makes This Different

Unlike many FPS assets that use scattered scripts with tangled dependencies, every system in this framework follows the same hierarchical state machine pattern. Player movement, cameras, and guns all use root states and sub states managed through a StateFactory. This means:

Consistency: Learn the pattern once, apply it everywhere. Adding a new player state, camera mode, or gun behavior follows the same steps.

Modularity: States are self-contained C# files. You can add, remove, or modify states without breaking others.

Readability: The state machine structure makes it easy to trace what's happening at any point. The player is always in exactly one root state and one sub state.

Demo Content vs. Framework

The asset includes demo weapons (Pistol, Revolver, Shotgun, SMG) and a Playground scene with example pickups and targets. These are examples showing how to use the framework. You are expected to replace or modify the demo content with your own models, animations, and configurations. The underlying systems (state machines, Scriptable Object architecture, save system) are the reusable framework.

2. Requirements & Compatibility

Requirement	Details
Unity Version	Unity 6 (6000.x) or later
Render Pipeline	Universal Render Pipeline (URP) recommended. Compatible with Built-in with minor adjustments.
Input System	com.unity.inputsystem — Required. Uses the new Input System, not the legacy Input Manager.
Cinemachine	com.unity.cinemachine — Required for the camera system.
Animation Rigging	com.unity.animation.rigging — Required for IK-based arm and weapon positioning.
TextMeshPro	com.unity.textmeshpro — Required for UI elements.
Platform	Windows, macOS, Linux (standalone). Console/mobile untested.

> **Note on Packages**
>
> All required packages should be included or auto-resolved when you import the asset. If any are missing, install them via Window > Package Manager.

3. Installation & Quick Start

Step 1: Import the Package

Import the Burntwax FPS Framework from the Asset Store via Window > Package Manager > My Assets. Let Unity resolve all dependencies.

Step 2: Open the Main Menu Scene

Navigate to the Scenes folder inside the Burntwax FPS Demo directory and open the **Init Menu** scene. Do not start with the Playground scene directly. The Init Menu handles initialization and scene loading.

Step 3: Press Play

Hit Play in the Editor. You should see the main menu. Start a new game to load into the Playground scene, where you can move around, pick up weapons, and test all the systems.

Step 4: Explore the Playground

The Playground scene is your sandbox. Walk around, pick up guns off the ground, shoot targets and crates, and try wall running on the designated surfaces. This scene contains examples of every system in the framework.

> **Important: Build Your Game in the Playground Scene**
>
> We recommend turning the Playground scene into your actual project scene rather than trying to drop the Player and Camera prefabs into an existing scene. The prefab hierarchy has specific parent-child relationships, script references, and layer settings that are easiest to maintain when you start from the included scene and modify it. For later scenes, just copy the GameObjects from Playground and place them in the new scene (see **Testing Scene**)

Step 5: Experiment with Variables

With the scene running, select the Player or Cameras GameObjects in the Hierarchy and adjust their exposed variables in the Inspector. Movement speed, jump height, camera sensitivity, and many other values are exposed for live tweaking.

✓ **Checkpoint:** Press Play from the Main Menu scene. You should be able to walk, jump, pick up at least one gun, and shoot it at a target. If this works, your installation is correct.

4. Folder Structure

Here is the layout of the main asset directory. Familiarize yourself with this before making changes. It will help you find scripts, prefabs, and configurations quickly.

```
Assets/
  Burntwax Collective/
    Burntwax FPS Demo/
      Scenes/
        Main Menu ............... Entry point scene
        Playground .............. Demo sandbox scene
      Scripts/
        Player/
          Movement/ ............. All player state scripts
            PlayerStateMachine.cs
            PlayerStateFactory.cs
            PlayerBaseState.cs
            PlayerGroundedState.cs, PlayerJumpState.cs, etc.
          Input/
            Manager/InputManager.cs
            PlayerInput ........ Input Actions Asset
        Camera/ ................. Camera state machine scripts
        Guns/ ................... Gun state machine scripts
          GunStateMachine.cs, GunType.cs, ProceduralRecoil.cs
        SaveLoad/ObjectManager.cs
        Effects/BulletSurfaceManager.cs
      Guns/ .................... Scriptable Objects for weapons
        Ammo/, Audio/, CamRecoil/, Crosshair/, Damage/, Shoot/, Trail/
        Pistol, Revolver, Shotgun, SMG
      SurfaceEffects/Bullet/ ... Impact effect Scriptable Objects
        Crate, Glass, Ground, Metal, Rock, Target, Wall
      Burntwax.C...asmdef ...... Assembly Definition file
```

> **Assembly Definition**
>
> The project uses an Assembly Definition file (.asmdef) to manage script compilation. If you install a new Unity package and get missing reference errors when using it in code, you will need to add that package to the Assembly Definition References. See Troubleshooting for details.

5. Architecture Overview

The framework is built on one core pattern used everywhere: the hierarchical state machine. Understanding this pattern is the single most important thing for working with this asset.

The Hierarchical State Machine Pattern

A state machine is a way of organizing code so that an object (like the player) is always in exactly one "state" at a time, and each state defines its own behavior. A hierarchical state machine adds layers: the player is simultaneously in a root state (the broad category of what they're doing) and a sub state (the specific action within that category).

For example: a player who is on the ground and running is in the root state Grounded and the sub state Sprint. If they jump while sprinting, the root state switches to Jump but the sub state can remain Sprint (they're still holding the sprint input mid-air).

System Architecture

All three major systems share the same structure:

InputManager — Receives all player input via the Input System and publishes events to subscribers.

PlayerStateMachine — Root states: Grounded, Jump, Fall, WallRun, Slope. Sub states: Idle, Walk, Sprint, Crouch.

CamStateMachine — Root states: FPS Cam, Aim Cam, and any custom cameras you add.

GunStateMachine — Root states: Idle, Shoot, Reload, Equip, Stow, Empty. Configured via Scriptable Objects.

BulletSurfaceManager — Tag-based impact effects using Scriptable Objects.

ObjectManager — Save/Load system that tracks objects by layer.

How the Three State Machines Connect

All three state machines (player, camera, gun) share the same code structure. Each one has:

A StateMachine script — lives on the main GameObject. Contains shared variables, runs constant-update functions (like ground detection), and holds references to the current root and sub states.

A StateFactory script — creates and returns state instances. This is where you register new states.

Individual State scripts — one C# file per state. Each implements EnterState(), UpdateState(), ExitState(), and CheckSwitchStates().

The camera state machine is connected to the player state machine, so player movement states can trigger camera state changes (e.g., aiming reduces FOV). The gun state machine operates independently but reads input from the same InputManager.

What Skill Level Do I Need?

Task	Skill Level	What You Need to Know
Tweaking movement/gun values	Beginner	Unity Inspector basics
Creating a new gun	Beginner	Scriptable Objects, Unity Animator basics
Changing input bindings	Beginner	Unity Input System basics
Adding a new player state	Intermediate	C# scripting, state machine concepts
Adding a new camera mode	Intermediate	Cinemachine basics, C# scripting
Extending the save system	Intermediate	C# scripting, Unity layers
Swapping out the arm model	Advanced	Animation Rigging, IK constraints, Blender/FBX workflow

6. Player Movement

6.1 State Machine Concepts

The player is always in exactly two states at the same time: one root state and one sub state.

Root States

Root State	When Active
PlayerGroundedState	Player is on flat ground
PlayerJumpState	Player has just jumped and is ascending
PlayerFallState	Player is falling (not grounded, not jumping)
PlayerSlopeState	Player is on an angled surface
PlayerWallRunState	Player is running along a wall

Sub States

Sub State	When Active
PlayerIdleState	No movement input
PlayerWalkState	Movement input at normal speed
PlayerSprintState	Movement input + sprint held
PlayerCrouchState	Crouch input held

Sub states can apply across multiple root states. For example, you can be sprinting while grounded, sprinting while wall running, or sprinting while falling (maintaining momentum). You can define exceptions per state — for example, preventing crouching during the jump or fall root states.

Code Difference: Root vs. Sub State

The only code difference between a root state and a sub state is a single line in the constructor:

```
// Root state — sets IsRootState = true
public PlayerGroundedState(...) : base(...)
{
    IsRootState = true;
}
```

```
// Sub state — does NOT set IsRootState
public PlayerSprintState(...) : base(...)
{ }
```

Everything else, EnterState(), UpdateState(), ExitState(), CheckSwitchStates(), works identically.

The PlayerStateMachine Script

PlayerStateMachine is the central script attached to the Player GameObject. It holds shared variables (movement speed, gravity, ground check results) and runs functions that need to update every frame regardless of state, such as ground detection and slope angle checking. Think of it as the "brain" that all states can read from and write to.

6.2 Adding a New State

Let's walk through adding a hypothetical PlayerSlideState as a new root state.

Step 1: Register the state in PlayerStateFactory. Open PlayerStateFactory.cs and add your new state to the enum list. Then create its context and factory function following the pattern of the existing states.

Step 2: Create the state script. Create a new C# file called PlayerSlideState.cs. Use any existing root state as a template. Your constructor should set IsRootState = true.

Step 3: Implement the state logic. Fill in the four key methods:

Method	Purpose
EnterState()	Runs once when the player enters this state. Set up slide velocity, play animation, etc.
UpdateState()	Runs every frame while in this state. Apply slide movement, check speed thresholds, etc.
ExitState()	Runs once when leaving this state. Clean up, reset variables.
CheckSwitchStates()	Runs every frame. Check conditions for transitioning to another root state.

Step 4: Add transition logic. In the states you want to transition from (e.g., PlayerGroundedState), add a condition in CheckSwitchStates() that switches to your new slide state when the right input/conditions are met.

✓ **Checkpoint:** Create a new state, register it in the factory, add transitions. Press Play. Your state should activate when its conditions are met. Add a Debug.Log in EnterState() to verify.

7. Cameras

The camera system uses Cinemachine virtual cameras (vcams) managed by the same hierarchical state machine pattern as player movement.

Included Cameras

Camera	Purpose
FPS Cam	Default first-person view with standard field of view
Aim Cam	Aiming mode with reduced FOV for precision shooting

Because the camera system uses the same state machine architecture as player movement, camera states can be tied directly to player states. For example, when the player enters the aiming sub state, the camera automatically transitions to the Aim Cam with its reduced FOV.

Adding a New Camera

Step 1: Create a new Cinemachine virtual camera in the scene and configure it with the settings you need (FOV, follow target, noise, etc.).

Step 2: Attach the new vcam reference to the CamStateMachine script on the Player GameObject.

Step 3: Open the camera StateFactory and add your new camera state, same process as adding a player state. Register it in the enum, create the context and function.

Step 4: Create your camera state C# file. In EnterState(), enable your vcam (set its priority higher). In ExitState(), disable it. In CheckSwitchStates(), define when to transition back.

✓ **Checkpoint:** Your new camera should activate when its trigger condition is met. You should see a smooth Cinemachine blend from the previous camera to your new one.

8. Gun System

The gun system combines Scriptable Objects (for data and configuration) with the hierarchical state machine (for behavior). Creating a new gun involves four phases. Follow them in order.

> **Before You Begin**
>
> You will need: a gun 3D model (prefab), animations for equip/stow/shoot/reload, audio clips, and a muzzle flash VFX prefab. You can use placeholder assets initially and swap them later.

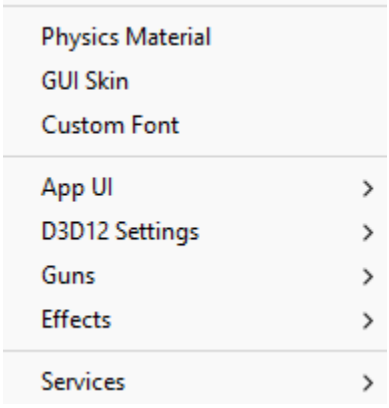
Phase 1: Create the Scriptable Object

The Scriptable Object is where all of your gun's data lives: its name, ID, type, and references to all configuration assets.

Step 1: Navigate to Assets > Burntwax Collective > Burntwax FPS Demo > Guns in the Project window.



Step 2: Right-click in the folder, then select Create > Guns (under the Services submenu). This creates a new Gun Scriptable Object.

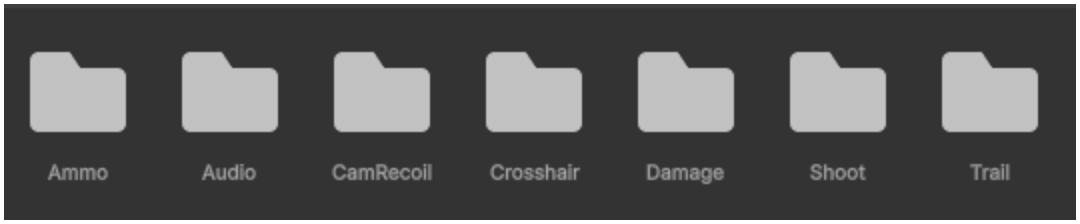


Step 3: Name it descriptively (e.g., "AssaultRifle").

Step 4: Set the ID. The ID determines the gun's position in the player's loadout. IDs are zero-indexed: 0 = first weapon slot, 1 = second, and so on. Set your new gun's ID to the next available number (e.g., 5). The ID must match the gun's position in the Ordered Guns list later.

Step 5: Set the Gun Type. Open GunType.cs and add a new entry to the enum (e.g., AssaultRifle). Then select this type in your Scriptable Object's dropdown.

Step 6: Create and assign configs. For each config slot on the Scriptable Object, create a corresponding config asset. Navigate to the config subfolders (Ammo, Audio, CamRecoil, Crosshair, Damage, Shoot, Trail) and right-click > Create > Guns > [Config Type]. You can duplicate existing configs and modify the values.



Config Reference

Config	What It Controls
Shoot Config	Fire rate, spread, bullets per shot, automatic vs. semi-automatic
Ammo Config	Magazine size, max reserve ammo, reload time
Audio Config	Shoot sound, reload sound, empty clip sound, equip/stow sounds
Damage Config	Base damage, damage falloff over distance, headshot multiplier
Trail Config	Bullet trail width, color, lifetime, material
Crosshair Config	Crosshair style, size, color, spread visualization
CamRecoil Config	Camera shake intensity, snappiness, return speed

Step 7: Assign each config to the corresponding slot on your Gun Scriptable Object in the Inspector. Note CamRecoil configs don't go in the gun Scriptable Object. CamRecoil will be attributed in later steps.



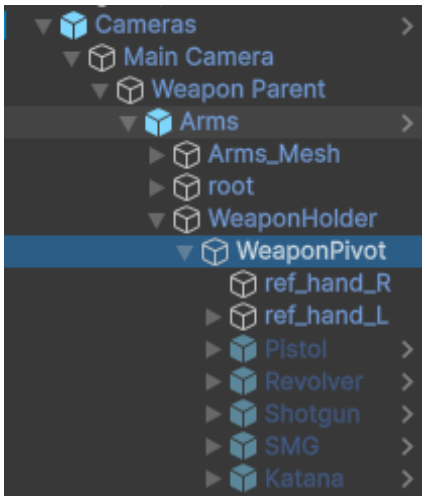
Step 8 (Optional): Enable specialized mechanics. Check Incremental Reload for shotgun-style reloading, or Procedural Recoil for SMG-style random recoil. See Section 8.6 for details.

✓ **Checkpoint:** You should have a named Gun Scriptable Object with an ID, a GunType enum entry, and all seven config slots filled. No missing references in the Inspector.

Phase 2: Set Up the Prefab in Scene

Now you'll place the gun model in the scene hierarchy so the player can hold and use it.

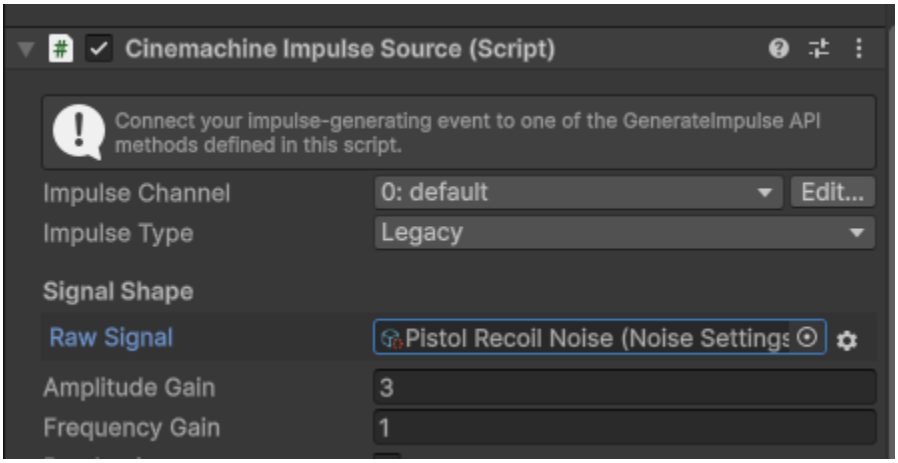
Step 1: In the scene Hierarchy, navigate to: Player > Cameras > Main Camera > Weapon Parent > Arms > WeaponHolder > WeaponPivot. You will see the existing guns listed here.



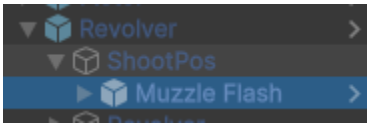
Step 2: Drag your gun prefab into this hierarchy as a sibling of the existing guns (directly under WeaponPivot).

Step 3: Zero out the position and rotation of your gun prefab. Set the scale to whatever looks correct for first-person view.

Step 4: Add the RigLayer component and the CinemachineImpulseSource component to your gun prefab. Here, you can add the CamRecoil Scriptable Object you created in Step 6.



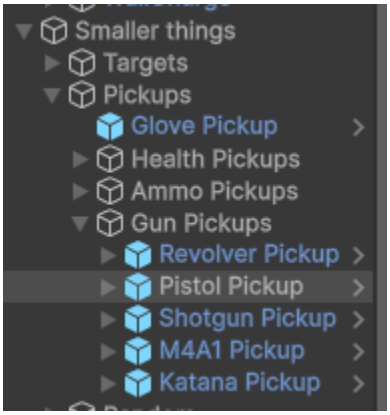
Step 5: Inside your gun prefab, create a child GameObject called ShootPos. Inside ShootPos, add your muzzle flash VFX prefab. This is where the gun's firing VFX will play from.



✓ **Checkpoint:** Your gun prefab should be in the Hierarchy under WeaponPivot, with zeroed position/rotation, RigLayer and CinemachineImpulseSource components, and a ShootPos child with a muzzle flash VFX.

Phase 3: Create the Pickup

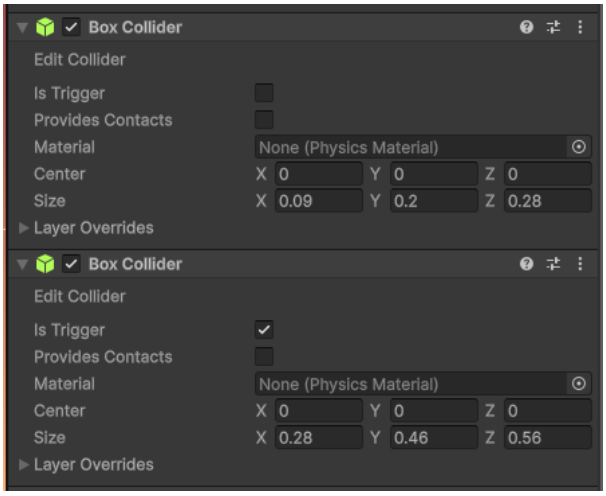
Guns are acquired through a pickup system. You need to create a world-space pickup prefab for your gun.



Step 1: Create a new prefab for your gun pickup. This is typically a simplified version of the gun model that sits on the ground in the world.

Step 2: Add the GunPickup script to the prefab (the same script used on the existing pickups).

Step 3: Add two colliders to the pickup prefab: one standard collider (Is Trigger = No) for physical collision, and one trigger collider (Is Trigger = Yes) for detecting when the player walks over it.



Step 4: In the pickup script's Inspector, assign your Gun Scriptable Object to the gun reference slot.

Step 5: Place your pickup prefab in the scene under the Pickups > Gun Pickups hierarchy.

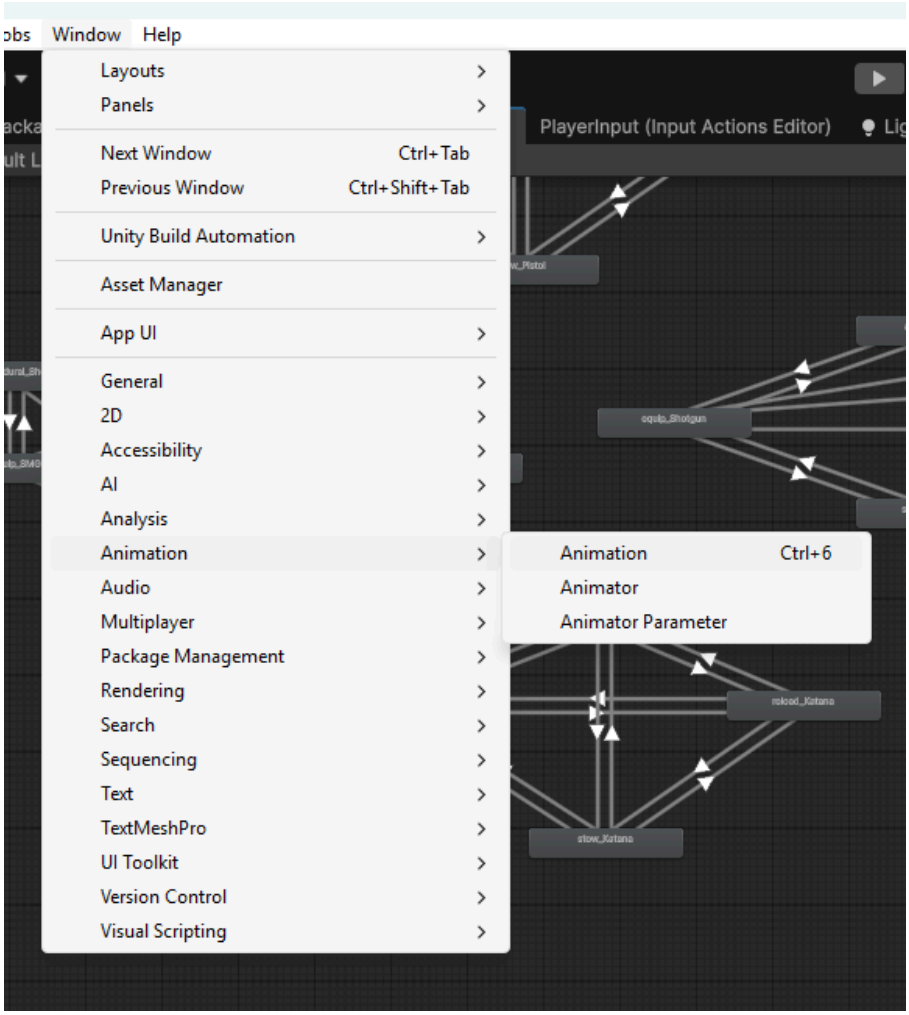
✓ **Checkpoint:** Walking into your gun pickup in Play mode should add the weapon to your inventory. You won't be able to use it properly yet until you wire up animations in Phase 4.

Phase 4: Wire Up Animations

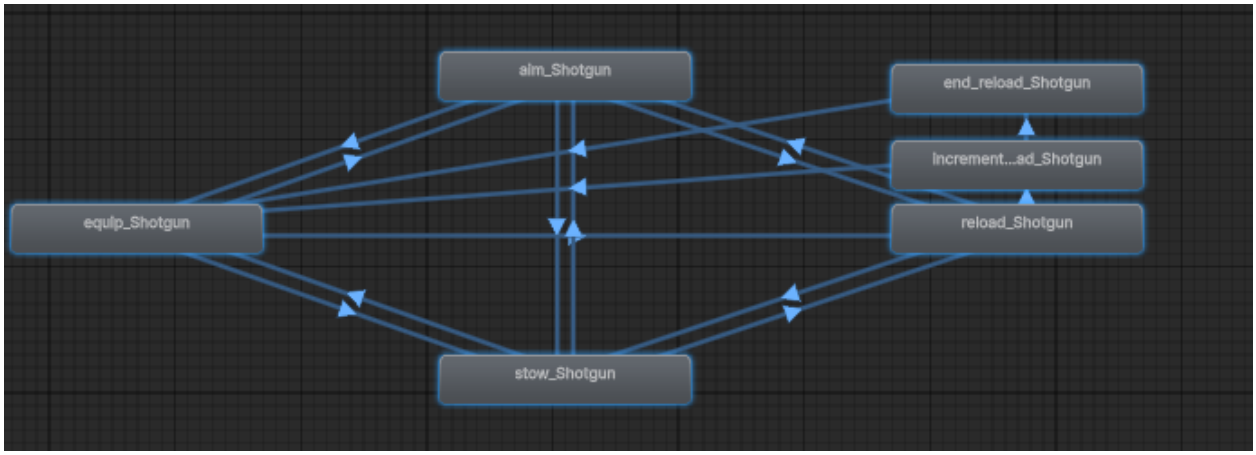
Step 1: Register the gun in the Ordered Guns list. Select the Arms GameObject (under Cameras > Main Camera > Weapon Parent > Arms). In the Inspector, find the Ordered Guns list. Add your gun at the index matching its Scriptable Object ID.

> **Critical: ID and List Order Must Match**
>
> The Scriptable Object ID and the position in the Ordered Guns list must be identical. If they don't match, the gun switching system will break. Selecting your gun will equip the wrong weapon, or fail entirely.

Step 2: Open the Animator. With the Arms GameObject selected, open the Animator window (Window > Animation > Animator). You will see the animation state machines for all existing guns.



Step 3: Copy an existing gun's animation structure. Choose the gun most similar to yours: Pistol/Revolver for standard magazine reload, Shotgun for incremental reload, SMG for procedural recoil. (**SEE 8.5**)

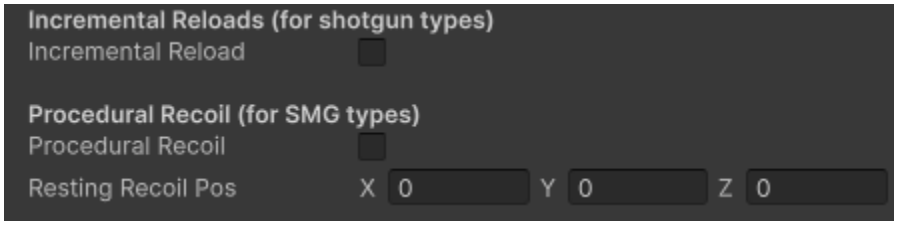


Step 4: Rename the animation states. Change all the state names to use your gun's GunType enum name. For example: aim_Shotgun becomes aim_AssaultRifle, equip_Shotgun becomes equip_AssaultRifle, and so on for every state.

Step 5: The animation clips from the copied gun will play as placeholders. You can replace them later by selecting each state in the Animator and swapping the Motion clip.

✓ **Checkpoint:** Press Play and pick up your new gun. You should be able to equip it, shoot it (with placeholder animations), reload it, and switch to/from other weapons. If all of that works, your gun is fully integrated.

8.5 Specialized Mechanics



Incremental Reloading (Shotgun-style)

When the Incremental Reload checkbox is enabled on a Gun Scriptable Object, the gun reloads one round at a time rather than swapping an entire magazine. The player can interrupt the reload at any point by firing. This requires the Shotgun-style animation state structure in the Animator.

Procedural Recoil (SMG-style)

When the Procedural Recoil checkbox is enabled, the gun uses code-driven recoil instead of animation-driven recoil. This creates a randomized kick pattern suitable for automatic weapons. The

Resting Recoil Position (X, Y, Z) defines where the gun returns to after each recoil kick. This uses the ProceduralRecoil script on the WeaponPivot GameObject.

9. Impact Effects

The impact effect system determines what happens visually and audibly when a bullet hits a surface. Different materials produce different effects: metal sparks, wood splinters, glass shatter sounds, etc. This system uses Scriptable Objects paired with Unity's Tag system.

Included Surface Effects

Surface	Tag	Typical Use
Crate	Crate	Wooden crates and boxes
Glass	Glass	Windows, glass panels
Ground	Ground	Floor surfaces, dirt, concrete ground
Metal	Metal	Metal walls, railings, pipes
Rock	Rock	Stone walls, boulders
Target	Target	Shooting targets
Wall	Wall	Generic wall surfaces

Creating a New Impact Effect

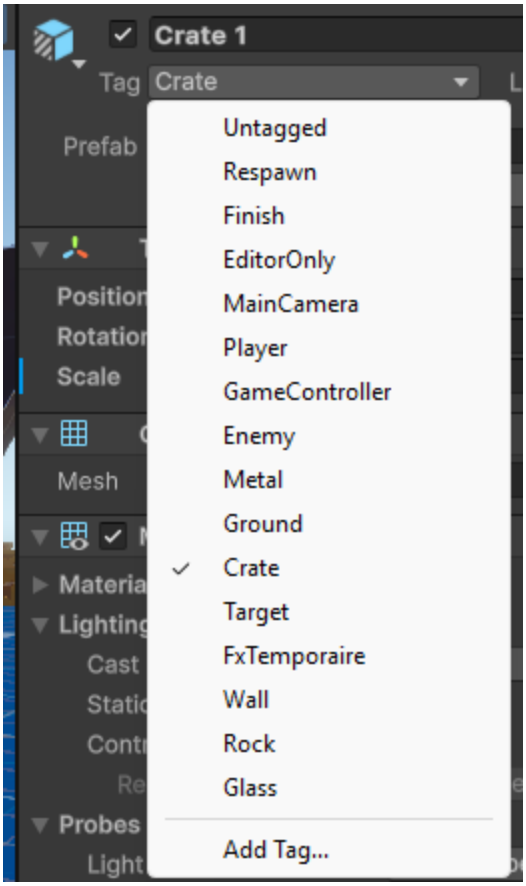
Step 1: Navigate to Assets > Burntwax Collective > Burntwax FPS Demo > SurfaceEffects > Bullet.



Step 2: Right-click > Create > Effects > Surface Effect. Name it (e.g., "Sand").

Step 3: In the new Scriptable Object, set the Surface Tag name (e.g., "Sand").

Step 4: Create a matching Tag in Unity. Select any GameObject, click the Tag dropdown, click Add Tag, and create a new tag with the exact same name.



Step 5: Assign a VFX prefab and a list of audio clips (the system picks one randomly each time).

Step 6: Add your new Surface Effect Scriptable Object to the list on the BulletSurfaceManager GameObject (found under Managers in the scene hierarchy).

Step 7: Apply the new tag to any GameObjects in your scene that should use this surface effect.

✓ **Checkpoint:** Shoot the tagged objects. You should see the correct VFX and hear a randomly selected audio clip from your list on each impact.

10. Save & Load System

Basic Usage

Action	Key
Save	F6
Load	F7

What Gets Saved

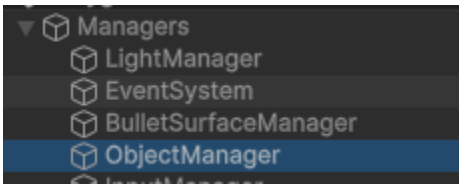
The save system tracks the state of objects in the scene based on their layer. Different layers save different data:

Object Type	Layer	Saved Data
Gun Pickups	**Consumable**	Position, rotation, velocity
Ammo Pickups	**Consumable**	Position, rotation, velocity
Health Pickups	**Consumable**	Position, rotation, velocity
Crates	**Destructable**	Position, rotation, health
Targets	**Basic**	Position, rotation

Player state (position, inventory, ammo counts) is also saved.

Adding New Saveable Objects

Step 1: Find the ObjectManager GameObject in the scene hierarchy (under Managers).



Step 2: In the Inspector, find the Prefab Configuration section. Add a new element to the Prefabs list.

Step 3: Assign two references: the Prefab Asset (the prefab to instantiate when loading) and the Parent (the parent GameObject in the hierarchy where it should spawn).



Step 4: Assign a layer to your new object. The ObjectManager uses layers to determine what data to save. Check existing examples to see which layer your object should use.

Basic: Simple objects that can change position during run-time, doesn't require saving velocity

Consumable: Objects that can move fluidly throughout the scene, but can be consumed by the player

Destructable: Like consumables, but have a measured health for destructible purposes

Step 5: If your object requires saving additional data beyond what existing layers support, add a new case in the ObjectManager script for your layer.

> **Editor vs. Build Save Conflicts**
>
> Save data created in the Unity Editor and save data created in a standalone build are stored separately. Always save fresh (F6) in whatever environment you're currently testing. A save from the Editor will not work in a build, and vice versa.

11. Adding New Arms (Advanced)

This section covers how to replace the default arm model with your own. The framework uses a **standardized bone naming convention** so that once you set up your arms correctly the first time, swapping to a different arm model in the future takes just a few minutes with no rig reconfiguration.

The guide is split into two parts: a **one-time setup** (11.1-3) where you can **establish a new standard skeleton for future use (not important unless you absolutely need to change the structure)**, and the **ongoing swap process** (11.4) which is what you'll use every time after that.

> **Prerequisites**
>
> Before starting, you should be comfortable with: Blender's Armature Edit Mode, basic weight painting, FBX export settings for Unity, and Unity's Animation Rigging package (Two Bone IK Constraints). If any of these are new, review Unity's official Animation Rigging documentation first.

Standard Bone Naming Convention

Every arm model used in this framework must follow this exact bone naming. The IK constraints in Unity reference these names directly. If they don't match, the rig breaks.

Role	Right Arm	Left Arm
IK Root	UpperArm_R	UpperArm_L
IK Mid	Forearm_R	Forearm_L
IK Tip	Hand_R	Hand_L
Thumb	Thumb1_R → Thumb2_R → Thumb3_R	Thumb1_L → Thumb2_L → Thumb3_L
Index	Index1_R → Index2_R → Index3_R	Index1_L → Index2_L → Index3_L
Middle	Middle1_R → Middle2_R → Middle3_R	Middle1_L → Middle2_L → Middle3_L
Ring	Ring1_R → Ring2_R → Ring3_R	Ring1_L → Ring2_L → Ring3_L
Pinky	Pinky1_R → Pinky2_R → Pinky3_R	Pinky1_L → Pinky2_L → Pinky3_L

Hierarchy rules: Forearm is a child of UpperArm. Hand is a child of Forearm. All finger root bones (Thumb1, Index1, etc.) are children of Hand. Finger bones chain sequentially (1 → 2 → 3).

> **No Extra Bones**
>
> Do not include shoulder bones, twist bones, or _end bones. The Two Bone IK system expects exactly three bones per arm chain: UpperArm, Forearm, Hand. Extra bones in the chain will break the IK constraints.

11.1 Prepare Your Arms in Blender

Step 1: Clean up the armature

Open your arms .blend file and select the Armature. Enter Edit Mode (Tab).

- Delete all _end bones: Select > All by Trait > Leaf Bones, then press X to delete. Verify you only deleted the _end bones, not actual finger tip bones.
- Delete any shoulder or twist bones if present. Your chain should start at UpperArm.

Step 2: Rename all bones

Select each bone and press F2 to rename. Use the exact names from the Standard Bone Naming Convention table above. Spelling and capitalization must match exactly.

Step 3: Verify the hierarchy

In the Properties panel (Bone tab → Relations → Parent), confirm the parent-child relationships match the spec. If a bone has the wrong parent, select the child, Shift-click the intended parent, and press Ctrl+P → Keep Offset.

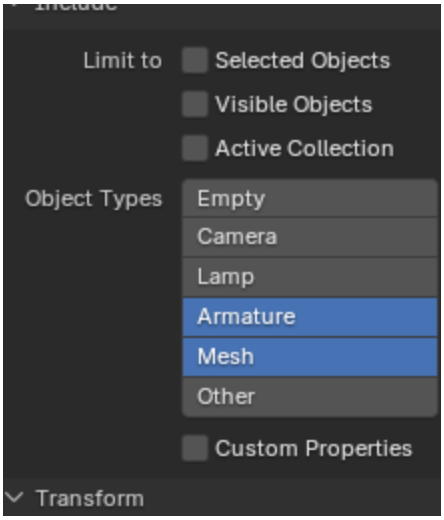
Step 4: Verify weight painting

Switch to your mesh object and enter **Weight Paint Mode**. Click through each bone in the Armature to confirm the vertex groups still show correct painting. Renaming bones in Blender automatically renames the corresponding vertex groups, but double-check that all vertex group names match the new bone names exactly. Fix any mismatches in Object Data Properties → Vertex Groups.

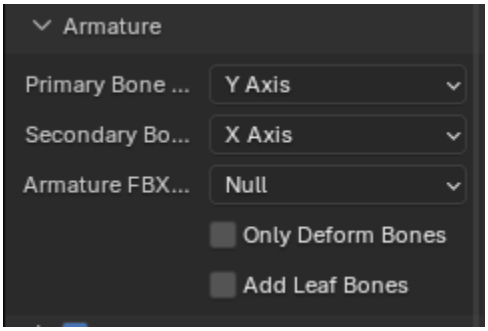
> **What is weight painting?**
>
> Weight painting defines which bones move which parts of the mesh. Each vertex has a weight from 0 (no influence) to 1 (full influence) for each bone. In Weight Paint Mode, red = full influence, blue = no influence.
>
> Blender's automatic weights (Armature Deform → With Automatic Weights) give a good starting point — you typically only need to touch up the wrist and elbow areas manually.

Step 5: Export as FBX

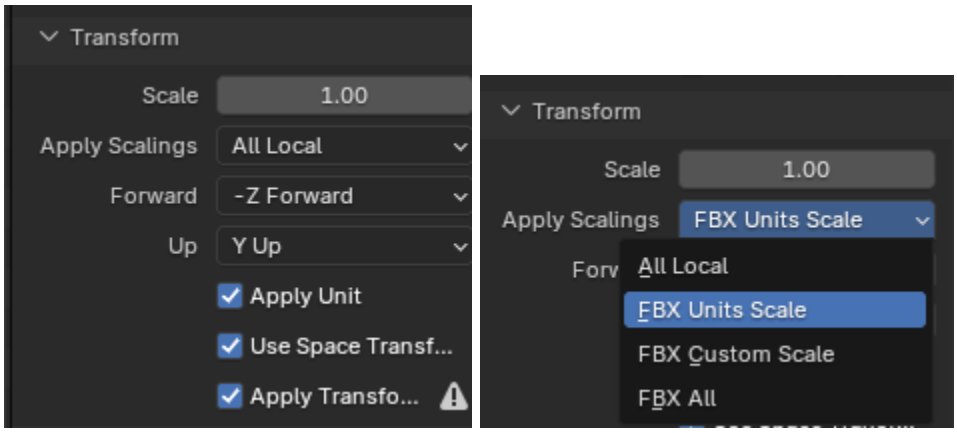
1. File → Export → FBX.
2. Select Armature and Mesh only



3. **Uncheck "Add Leaf Bones"** in the Armature section below. This is what prevents _end bones from reappearing in Unity.



4. Enable FBX Unit Scale and Apply Transform.



5. Export.

> **Critical: Add Leaf Bones**
>
> The "Add Leaf Bones" checkbox in Blender's FBX exporter is what creates _end bones in Unity. Always uncheck it. Every export, every time.

11.2 Set Up in Unity

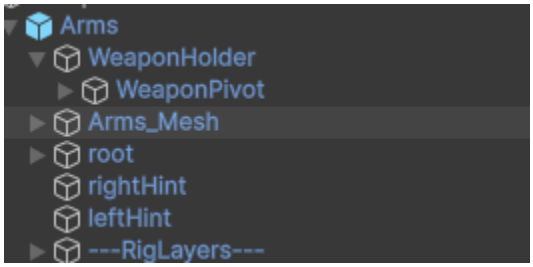
Step 6: Import the FBX

Drag your FBX into the Unity project. In Import Settings, set the Rig tab to Generic. On the Materials tab, fix materials if arms aren't visible. Hit Apply.

Step 7: Place in the scene hierarchy

Your existing hierarchy looks like this:

```
Weapon Parent
    └─ Arms          (Animator + Rig Builder)
        ├─ Arms_Mesh     (old mesh — remove this)
        ├─ root          (old armature — remove this)
```

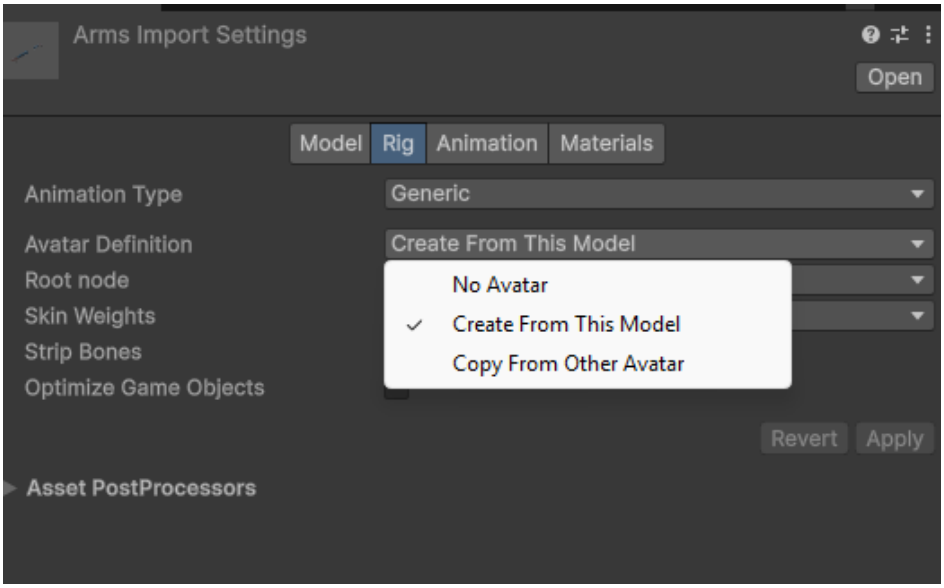


Delete or disable the old Arms_Mesh and old root (armature). Drag in the new FBX's mesh and armature as children of the Arms GameObject. Everything else stays exactly where it is.

Step 8: Update the Animator

Select the Arms GameObject. In the Animator component, assign the new Avatar that was auto-generated with your FBX import (it's a sub-asset of the FBX file in the Project window).

If it wasn't auto generated, then go to the fbx model in unity, and open up the inspector. You'll see a rig tab, and you can create a **Generic** Avatar from there. Do not create a Humanoid Avatar.

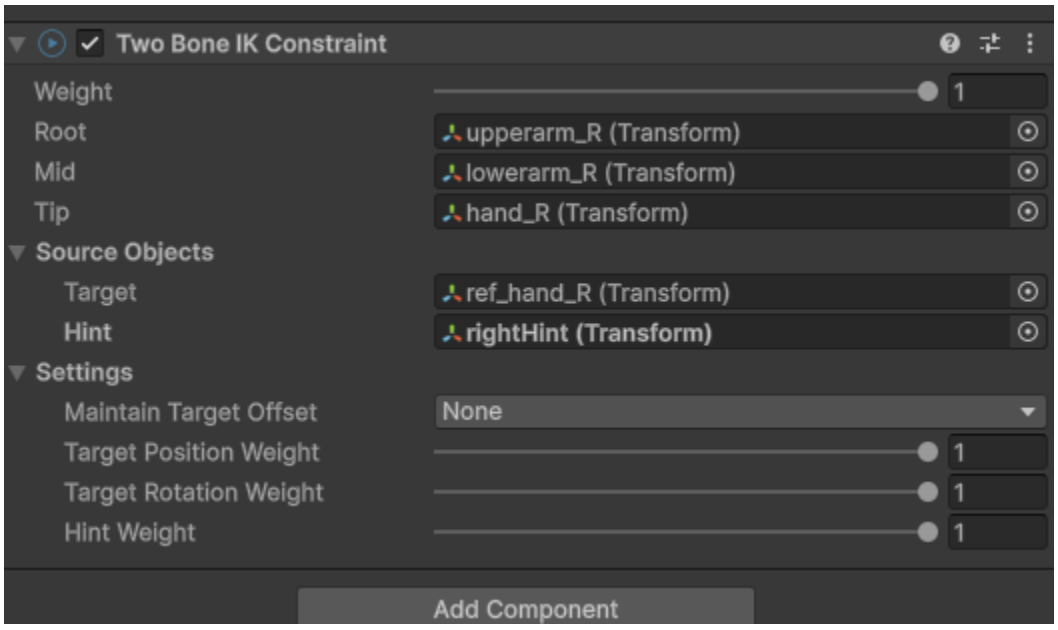


Step 9: Wire up the IK Constraints

This is the only rig work required. You need to assign 6 fields total:

Constraint	Root	Mid	Tip
RightHandIK	UpperArm_R	Forearm_R	Hand_R
LeftHandIK	UpperArm_L	Forearm_L	Hand_L

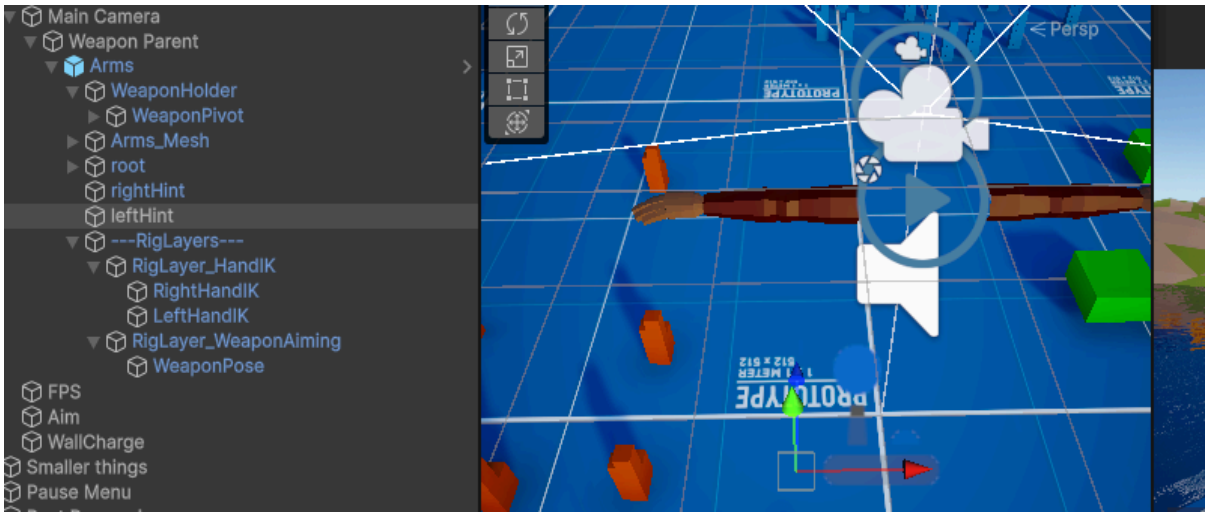
The Target fields should still point to ref_hand_R and ref_hand_L respectively. Do not change those.



Step 10: Set up hint objects

If you don't already have elbow hint objects, create them now. These prevent the elbows from bending in unnatural directions.

- Create an empty GameObject as a child of each forearm bone.
- Position it slightly behind the elbow, in the direction you want the elbow to point (usually backward).
- Assign each one to the Hint field on the corresponding Two Bone IK Constraint.



Step 11: Test

- Enter Play Mode. Drag WeaponPivot in the scene and the arms should follow.
- Pick up a gun. Both hands should grip it correctly.
- Switch weapons. Hands should reposition appropriately.

If arms don't move, check that all IK constraint weights are set to 1 and that the Rig Builder component is enabled on the Arms GameObject.

> ✓ **Checkpoint:** Dragging ref_hand_R and ref_hand_L in the Scene view should move the arms. Picking up a gun should result in both hands gripping it correctly, and switching weapons should reposition the hands.

11.3 Save the Template

This step makes all future arm swaps trivial. You're creating a reusable starting point that enforces the bone naming standard.

Step 12: Create a template .fbx file

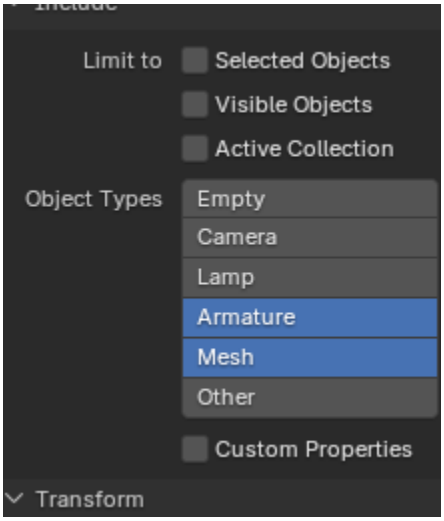
1. In Blender, open your cleaned-up arms file.
2. **File → Export → "arm name".fbx**. Use the export settings from above
3. Delete the mesh. Keep only the armature with the standardized bone names.
4. Save. This bare skeleton is your template. Anyone making new arms starts from this file.

✓ **Checkpoint:** You should have: working arms in Unity with the standardized bone names, and an .fbx file containing just the skeleton. Everything below is the process you'll repeat for every future arm swap.

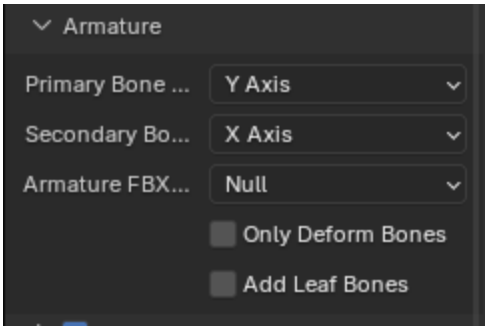
11.4 Swapping Arms (Ongoing)

From this point forward, replacing the arms with a completely new model is a **6-step process** with no rig reconfiguration.

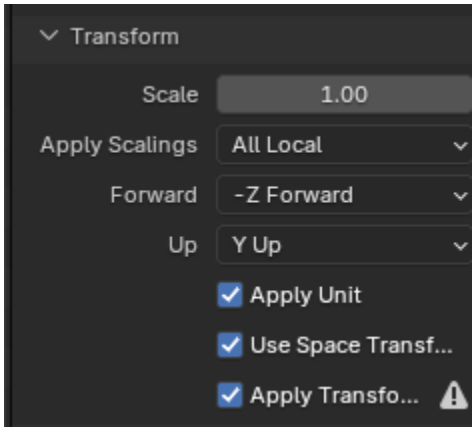
1. **Open ArmatureTemplate.fbx** in Blender. This gives you the standardized skeleton.
2. **Model and weight paint** your new arms onto the existing bones. Do not rename, add, or remove any bones.
3. File → Export → FBX.
4. Select Armature and Mesh only

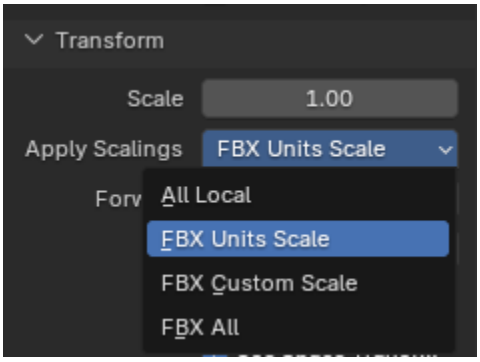


5. **Uncheck "Add Leaf Bones"** in the Armature section below. This is what prevents _end bones from reappearing in Unity.

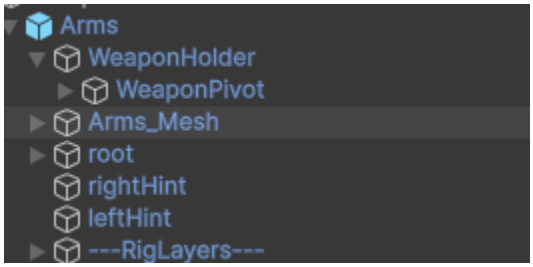


6. Enable FBX Unit Scale and Apply Transform.

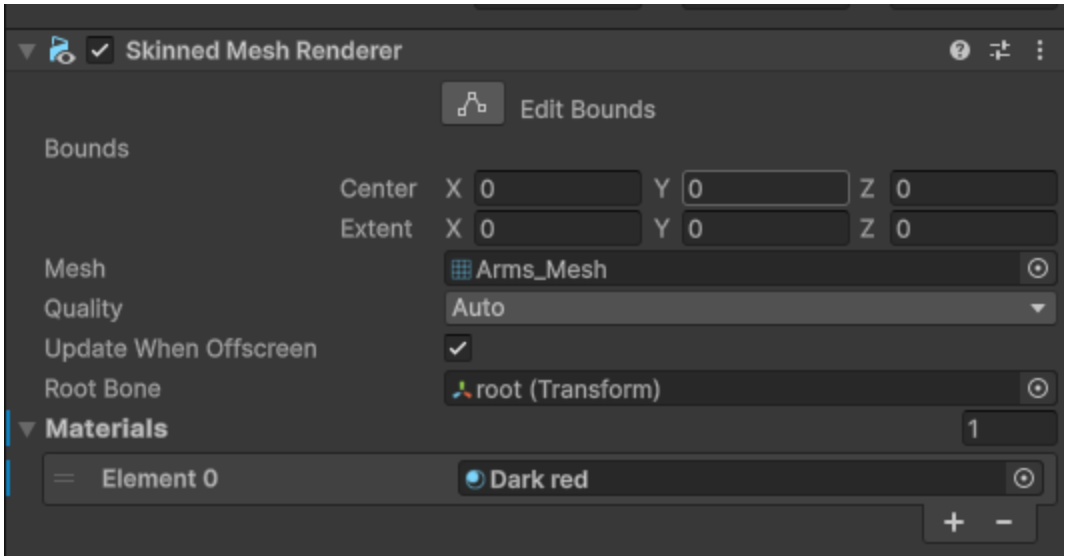




7. Export.
8. Import into Unity.
9. **In the Arms hierarchy:** remove the old Arms_Mesh and root. Add your new fbx to the scene and take the new Arms_Mesh + root, drop in them as children of the Arms GameObject.



For each mesh you import, make sure to go in ihe Skinned Mesh Renderer, enable “Update When Offscreen.”



Critical: Don’t Modify the Skeleton

The entire system depends on consistent bone names. If you rename, add, or remove bones from the template, you’ll need to reassign the IK constraints in Unity (Step 9). The template exists specifically so you never have to do this again.

Troubleshooting

“Arms don’t move when dragging ref_hand targets”

The IK constraints lost their bone references. Reassign Root, Mid, and Tip to the correct bones from your new armature (see Step 9).

“Arms disappear at certain camera angles”

Select the arms mesh. In the Skinned Mesh Renderer, enable “Update When Offscreen.” Unity’s frustum culling can incorrectly hide first-person arms.

“Fingers aren’t posing correctly”

Check that vertex group names on the mesh match the bone names exactly. Names are case-sensitive.

“Elbows bending the wrong way”

Reposition or recreate the hint objects (see Step 10). The hint tells the IK solver which direction the elbow should point.

“_end bones reappeared in Unity”

You forgot to uncheck “Add Leaf Bones” in Blender’s FBX export settings. Re-export with it unchecked.

“New arms clip through weapons”

Your new arm model has different proportions than the original. Adjust the positions of ref_hand_R and ref_hand_L to fit the new hand size and grip.

12. Modifying Inputs

Changing Existing Bindings

Step 1: Navigate to Assets > Burntwax Collective > Burntwax FPS Demo > Scripts > Player > Input > Manager.



Step 2: Double-click the PlayerInput asset. This opens the Input Actions Editor with all bindings.



Step 3: Click any binding to change its key/button assignment. Save when done.

Adding New Inputs

Step 1: Add a new Action in the Input Actions Editor.

Step 2: Open InputManager.cs and create subscribe/unsubscribe methods for your new action. Follow the pattern of the existing input methods, each action has a method that subscribes to the performed and canceled callbacks. No deep coding knowledge required.

```
0 references | 0 Unity Message
private void OnEnable()
{
    input = new PlayerInput();
    input.OnFoot.Enable();
    // The increment "subscribes" to the fucntion and plus or minusing it prevents memory leaks
    input.OnFoot.Move.performed += SetMove;
    input.OnFoot.Move.canceled += SetMove;

    input.OnFoot.Look.performed += SetLook;
    input.OnFoot.Look.canceled += SetLook;

    input.OnFoot.Sprint.started += SetSprint;
    input.OnFoot.Sprint.canceled += SetSprint;

    input.OnFoot.Aim.started += SetAim;
    input.OnFoot.Aim.canceled += SetAim;
```

13. Pause Menu & UI

Action	Input
Open pause menu	Esc
Resume (keyboard)	Esc again
Resume (mouse)	Click the Resume button
Return to main menu	Click the Main Menu button

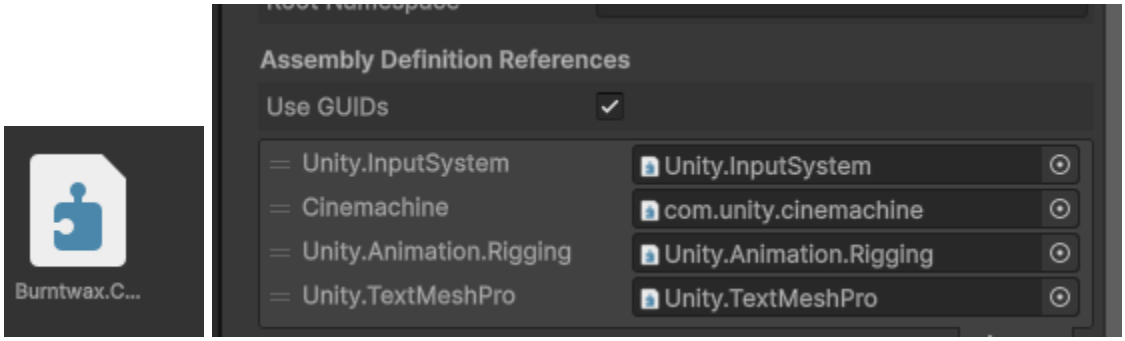
The pause menu currently supports resume and main menu navigation. Options and settings menus are planned but not yet implemented. If you need a settings menu, you can build one by adding UI elements to the pause menu canvas and hooking them up to your game settings.

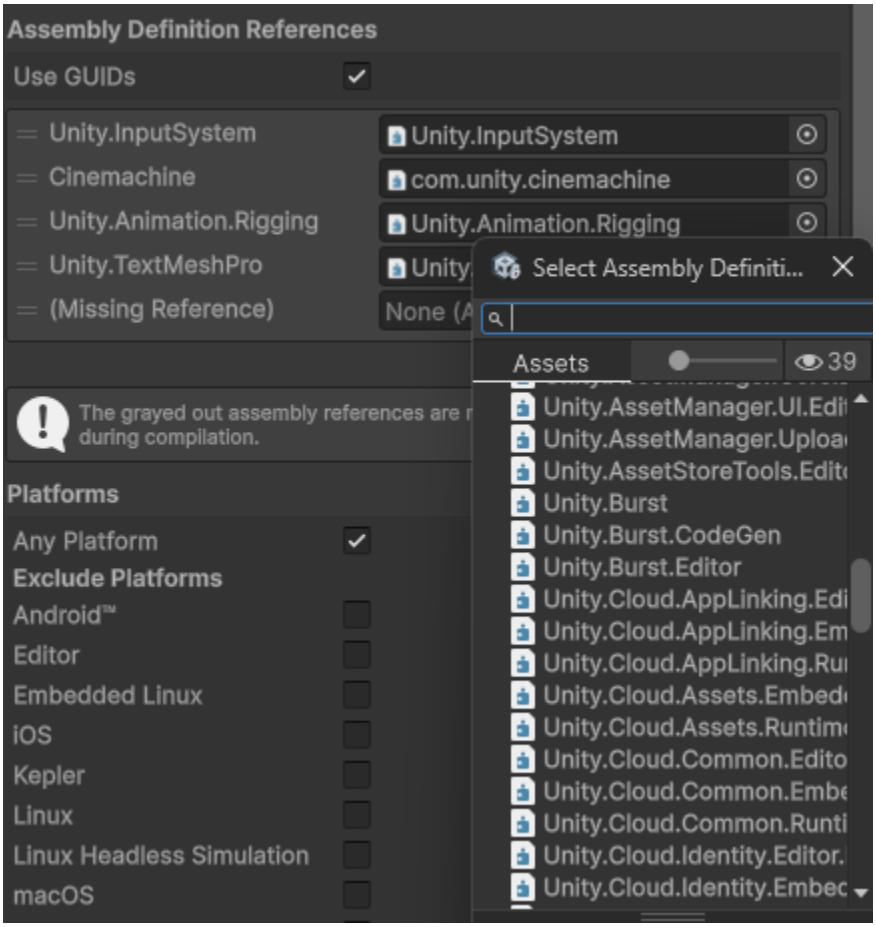
The UI uses Unity's standard Canvas system with TextMeshPro for text rendering. The pause system handles Time.timeScale and cursor lock state automatically.

14. Troubleshooting & FAQ

"I installed a new package and now I get missing reference errors"

The project uses an Assembly Definition file. When you add a new Unity package and reference it in code, you must add it to the Assembly Definition References. Find the .asmdef file in the Scripts folder, select it, and add your new package's assembly definition to the references list. Click Apply.



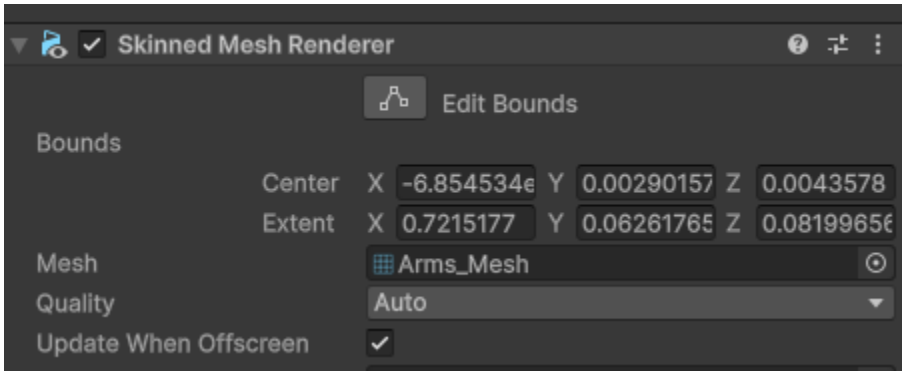


"My new gun doesn't switch properly / wrong gun equips"

The Scriptable Object ID and the position in the Ordered Guns list on the Arms GameObject must match exactly. Double-check both. Also verify that your GunType enum entry matches what's set on the Scriptable Object.

"Arms are invisible at certain camera angles"

Select the arms mesh and in the Skinned Mesh Renderer, enable Update When Offscreen. Unity's frustum culling can incorrectly hide the arms.



"Save/Load breaks when switching between Editor and Build"

Save files created in the Editor are separate from save files created in builds. Always save fresh (F6) in whatever environment you're currently testing.

"Objects I added don't save/load"

New objects must be registered in the ObjectManager's Prefab Configuration list with the correct layer assigned. The ObjectManager uses layers to determine what data to save.

"Seesaw or moving slopes don't work"

Objects that change their slope angle dynamically must be on the Variable Slope layer.

"Something isn't working and I can't figure out why"

Check Tags and Layers first. Many systems rely on correctly set tags (for impact effects) and layers (for save/load, ground detection, wall running, slope detection). If you add a new object that should behave like an existing one, make sure it has the same tag and layer.

15. Known Limitations

Limitation	Details
Options/Settings menu	Not yet implemented. Pause menu has a placeholder.
Mobile/Console	Untested on platforms other than Windows/macOS/Linux standalone.
Multiplayer	This is a single-player framework. Networking is not included.
Editor/Build saves	Save files do not transfer between Editor and standalone builds.

16. Glossary

Term	Definition
Scriptable Object	A Unity data container that stores configuration outside of scene objects. Used here for gun definitions, impact effects, and configs.
Hierarchical State Machine	A code pattern where an object has layered states: a root state (broad category) and a sub state (specific behavior), both active simultaneously.
Cinemachine	Unity's camera system package. Virtual cameras define camera behavior, and Cinemachine blends between them automatically.
vcam (Virtual Camera)	A Cinemachine component that defines camera settings. Multiple can exist; the one with highest priority is active.
Animation Rigging	Unity package for procedural animation. Used here for IK -> making arms reach for weapon grip points automatically.
Two Bone IK Constraint	An Animation Rigging component that makes a two-bone chain (like an arm) reach toward a target point.
Multi-Parent Constraint	An Animation Rigging component that makes one object follow another's position and rotation.
Rig Builder	The main Animation Rigging component. Attached to the root of an animated object, it evaluates all rig constraints in order.
Assembly Definition (.asmdef)	A Unity file that organizes scripts into compilation units. Requires explicit references to other packages your code depends on.
Input System (New)	Unity's modern input handling package, replacing the legacy Input Manager. Uses Input Action Assets for configurable bindings.
Root State	Top-level state describing the player's physical situation (Grounded, Jumping, Falling, Wall Running, On Slope).
Sub State	Secondary state describing what the player is doing within the root state (Idle, Walking, Sprinting, Crouching).

17. Support & Contact

[Update this section with your actual contact information before publishing.]

Channel	Details
Email	burntwaxcollective@gmail.com
Youtube	https://www.youtube.com/@BurntwaxCollective
Instagram	https://instagram.com/burntwaxcollective/
Discord	Coming Soon!

When reporting a bug, please include: your Unity version, a description of what you expected vs. what happened, and any console errors (copy the full error text).

18. Changelog

Version 1.0 — Initial Release

Hierarchical state machine framework for player movement, cameras, and guns. Five demo weapons (Pistol, Revolver, Shotgun, SMG). Scriptable Object-based gun creation and configuration system. Tag-based impact effect system. Save/Load system with ObjectManager. Input System integration with InputManager. Cinemachine camera system with FPS and Aim modes. Pause menu with resume and main menu. Demo Playground scene with pickups, targets, and varied terrain.